ME 461 LabVIEW #1
Due September 19th 2:00 PM

Introduction

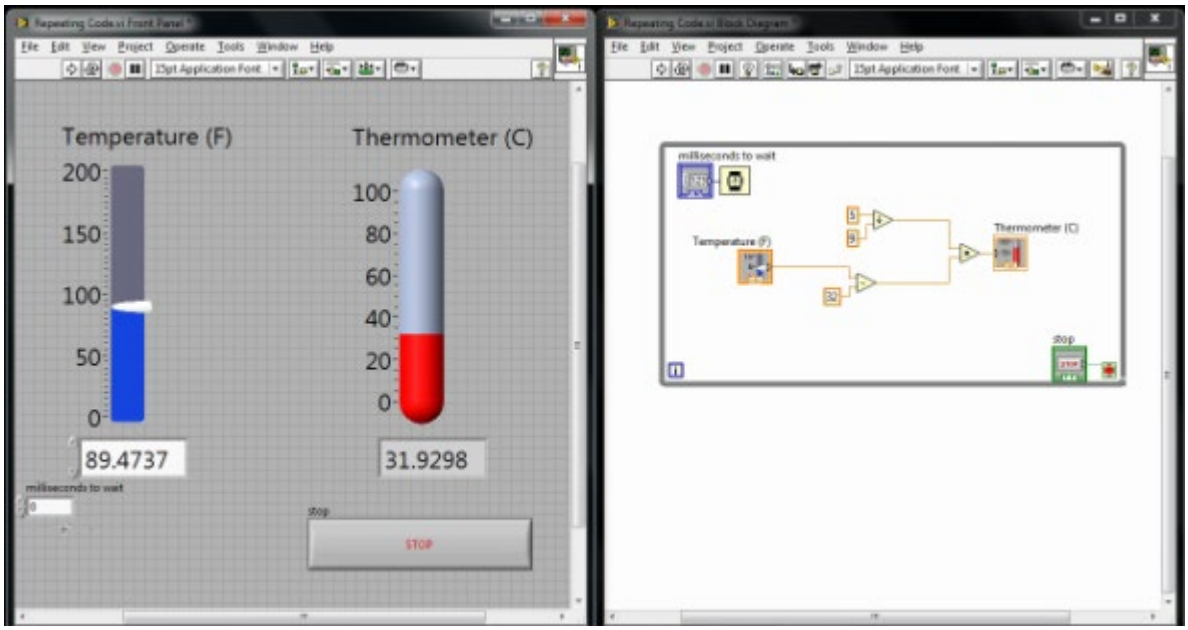
The goal of this assignment is to understand the basics of LabVIEW. Simply demonstrate your five LabVIEW programs working by the deadline. Grading for this assignment is simply full credit if you did the assignment and no credit if you did not complete the assignment. Make sure to ask questions if you get stuck.

Program 1

Read through at least the first two sections at the site: http://www.ni.com/gettingstarted/labviewbasics

Watch at least the first two videos and the tenth video at the site:
https://www.youtube.com/playlist?list=PLB968815D7BB78F9C

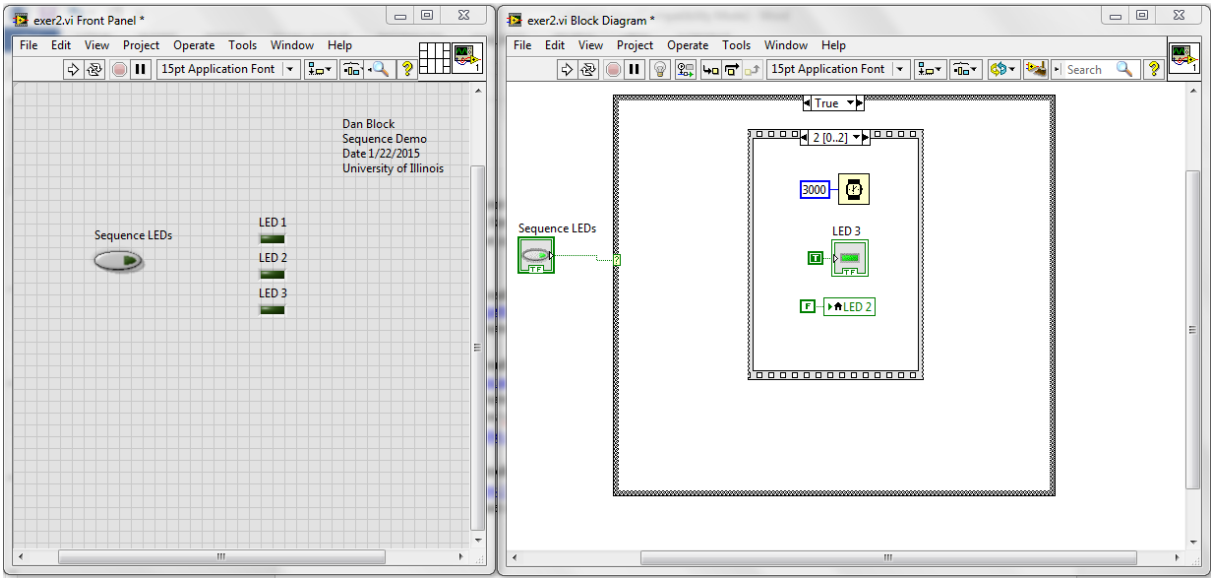
Then, reproduce (does not have to be exactly the same) the Fahrenheit to Celsius LABVIEW program that uses a loop structure to continuously run until a Stop button is pressed. Add some bells and whistles if you would like.



Program 2

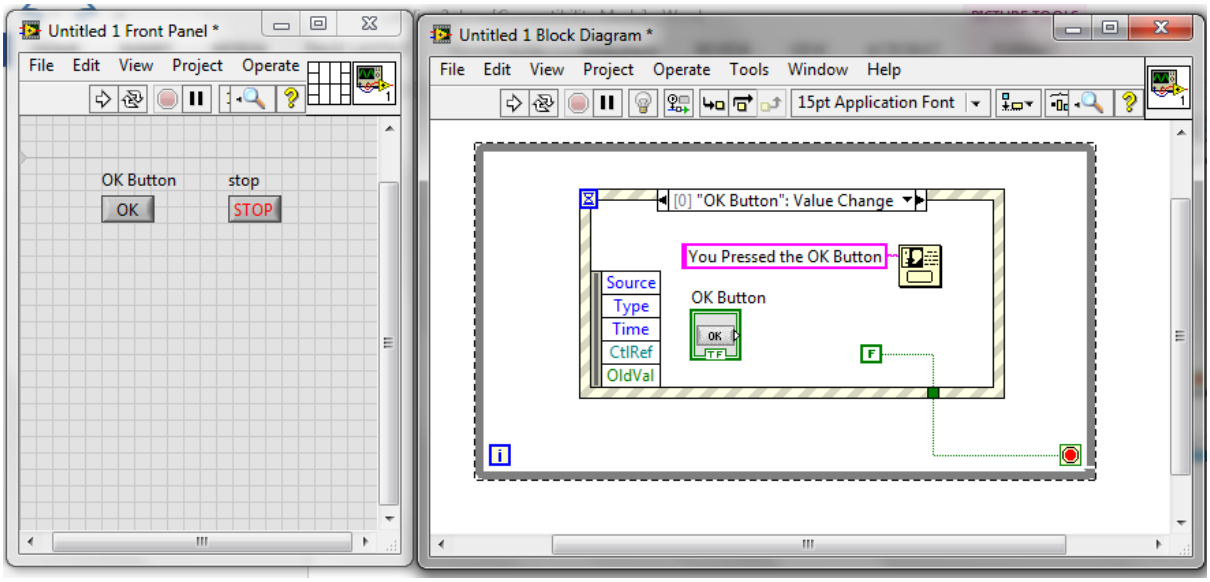
Next, do the following:

1. Read through all 12 sections at the site: http://www.ni.com/gettingstarted/labviewbasics
2. Watch the first 10 videos at the site: https://www.youtube.com/playlist?list=PLB968815D7BB78F9C
3. You can find other good YouTube videos. Here are a few others I found to get you started
    a. https://www.youtube.com/watch?v=Em5R_RM8E08
    b. https://www.youtube.com/watch?v=bflByHG5jdc
    c. https://www.youtube.com/watch?v=0Ea2IQeCIMY
    d. https://www.youtube.com/watch?v=QxoJljThkKk
4. Watch the following youtube video that introduces sequence structures: https://www.youtube.com/watch?v=DjN5Fpsjwng
5. To give you an introduction to sequence structures, reproduce the VI demonstrated in the YouTube video https://www.youtube.com/watch?v=03PykG1O1x0 . You may need to find some online help on “Case structures” as they are used in this video but not explained.



Program 3

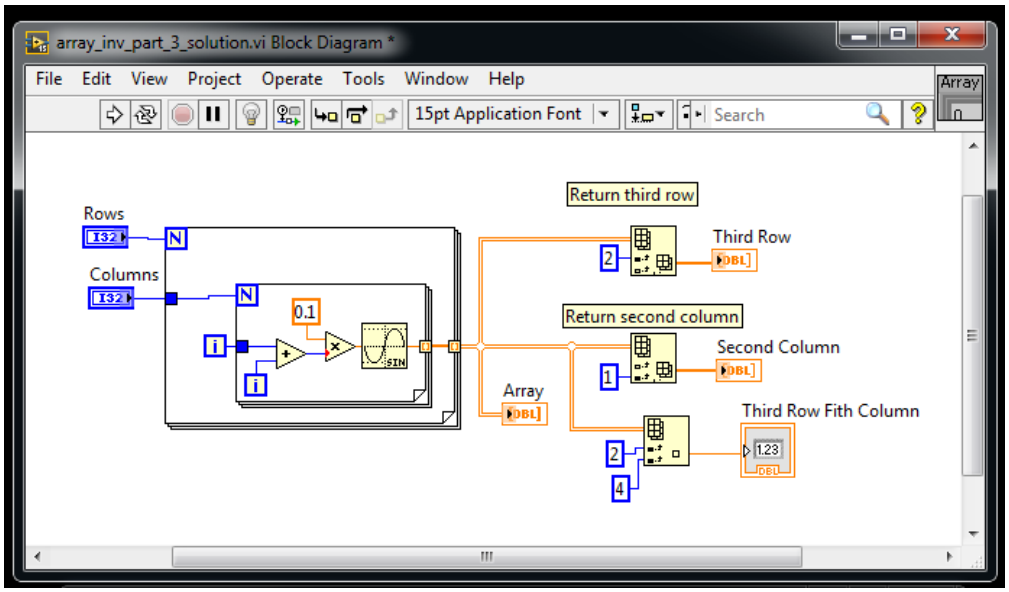
To give you an introduction to event structures, reproduce the VI demonstrated in the youtube video https://www.youtube.com/watch?v=8eO64fo3Pho . You do not need to demonstrate the initial "polling" VI. Just the event structure VI.

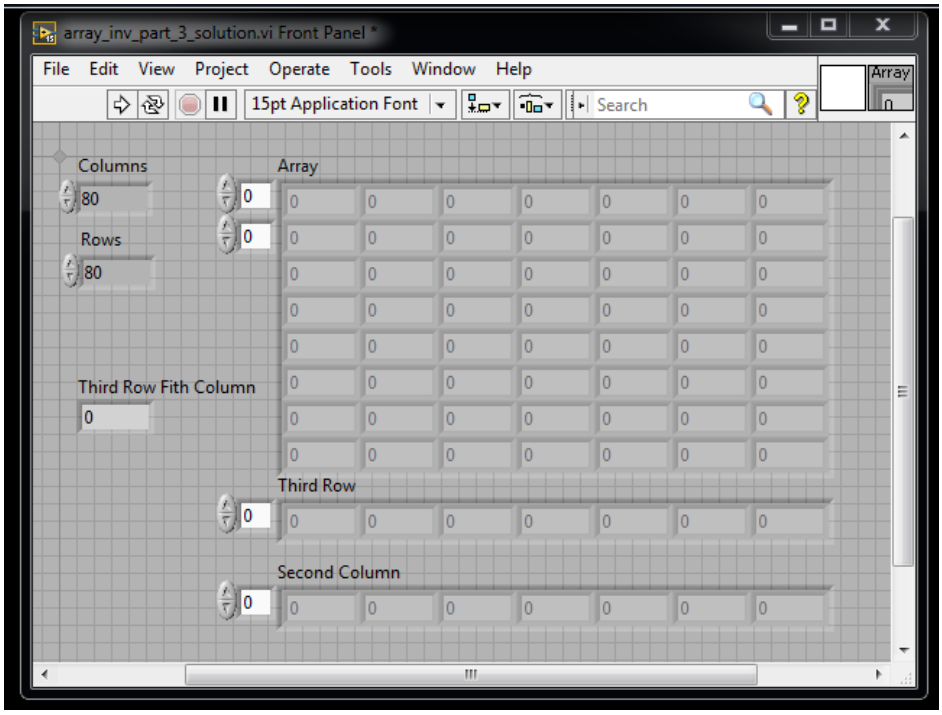


Programs 4 & 5

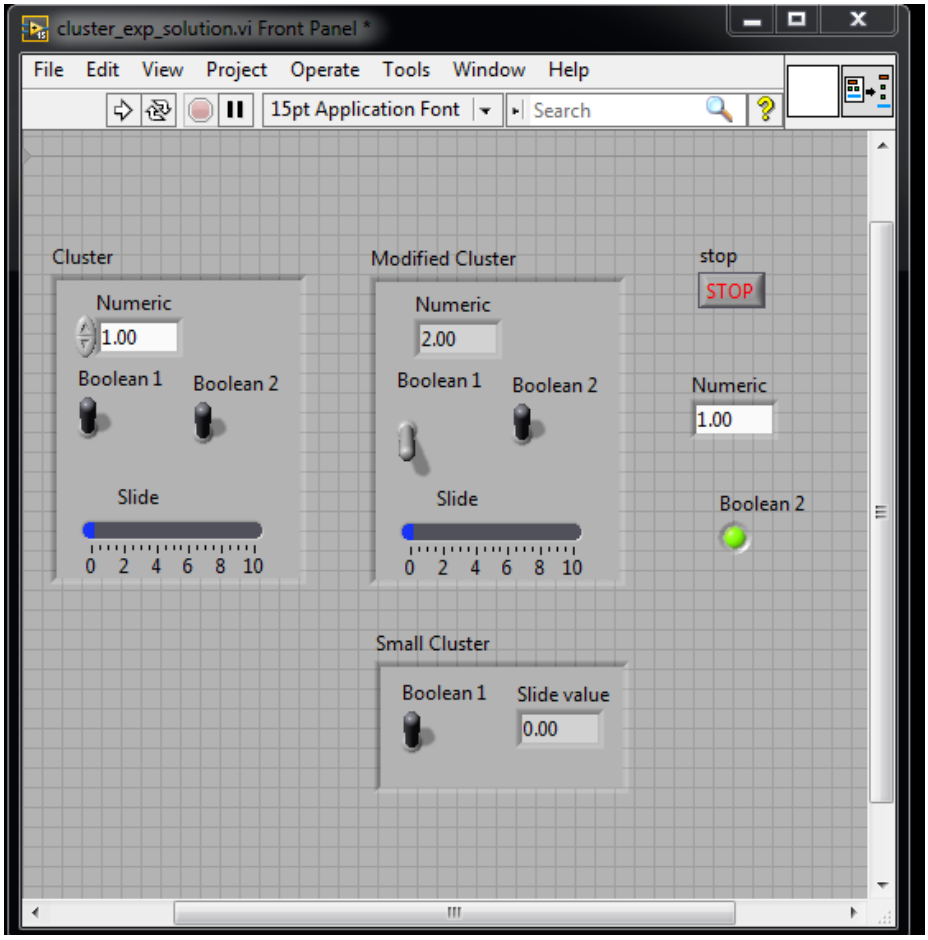
Read through the Array and Clusters Tutorial at https://www.youtube.com/watch?v=rzOT1zXBDiE and https://www.youtube.com/watch?v=_GlQ1riWjPc&list=PLB968815D7BB78F9C&index=8

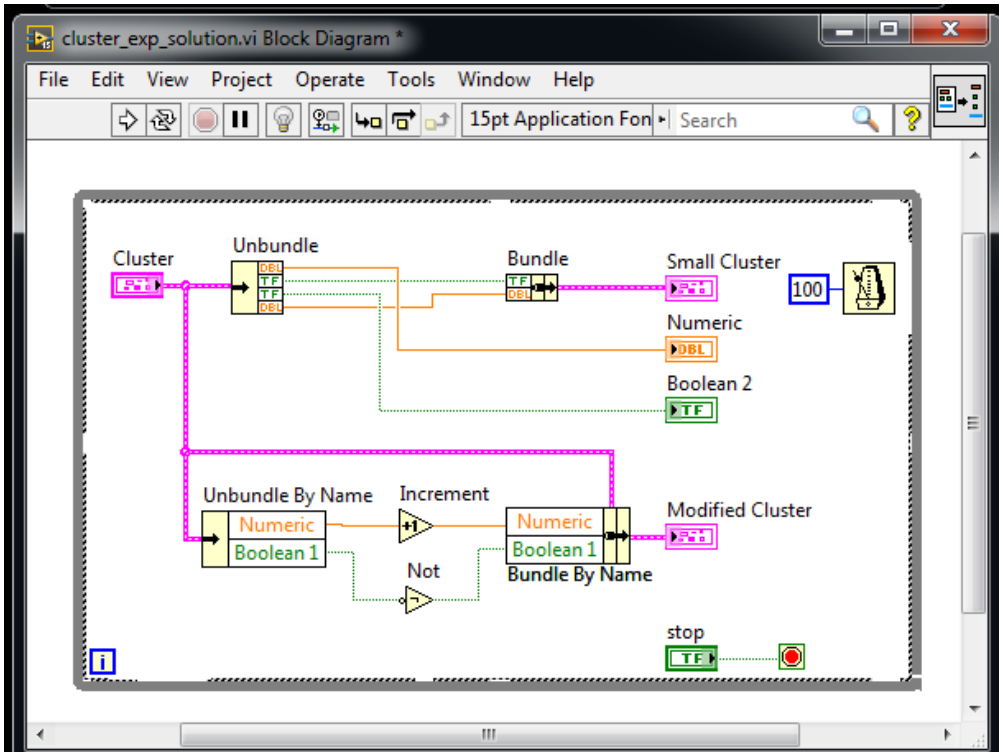
Then reproduce the following exercises. See how a "for loop" can create a multidimensional array and use the Index Array to pull out a single row and single column and a single element. Sine is found under Mathematics->Elementary->Trig





Create a Cluster and then produce two Clusters similar to the first.





LabVIEW Checklist

1. Fahrenheit to Celsius program
2. Case structure program
3. Button event program
4. Array example program
5. Cluster example program

ME 461 LabVIEW #2
Due October 10th 2:00 PM

Introduction

Note: There are a lot of small blocks in the LabVIEW code I give you below for Exercise 1 and 2. I have found if you open the Word document, you can zoom into the code pictures a bit better than the PDF file.

Exercise 1

First watch these two YouTube videos about shift registers (or find other tutorials online).

1. https://www.youtube.com/watch?v=wRvpb9jtRlE
2. https://www.youtube.com/watch?v=4ivYARn2fy8

For this assignment we are going to start working with the 2D Picture block in LabVIEW. After performing this exercise you should be able to use what you learn to create a 2D pictured environment for your robot and draw where the robot is located in its environment. Below is the LabVIEW program I would like you to reproduce for this assignment. It shows you how to draw lines in the 2D Picture. Each time through the while loop it takes a random number between 0 and 1 and draws a 25 pixel by 30 pixel rectangle (of course this could have been done easier by using the Draw Rectangle block but I wanted to show how to combine multiple draw commands.) Then, a shift register is used to keep track of all the lines printed in the 2D Picture as long as the Boolean LED is true. If you click on the Boolean LED making it false, the lines drawn in the previous loops should not be printed to the 2D Picture.

If your screen is big enough make the 2D Picture 800X800 pixels.

Location of the blocks that may be new to you for this assignment:

From the Front Panel:

Modern → Graph → Controls → 2D Picture

Modern → Boolean → Round LED

From the block diagram:

Programming → Graphics & Sound → Picture Functions → Move Pen

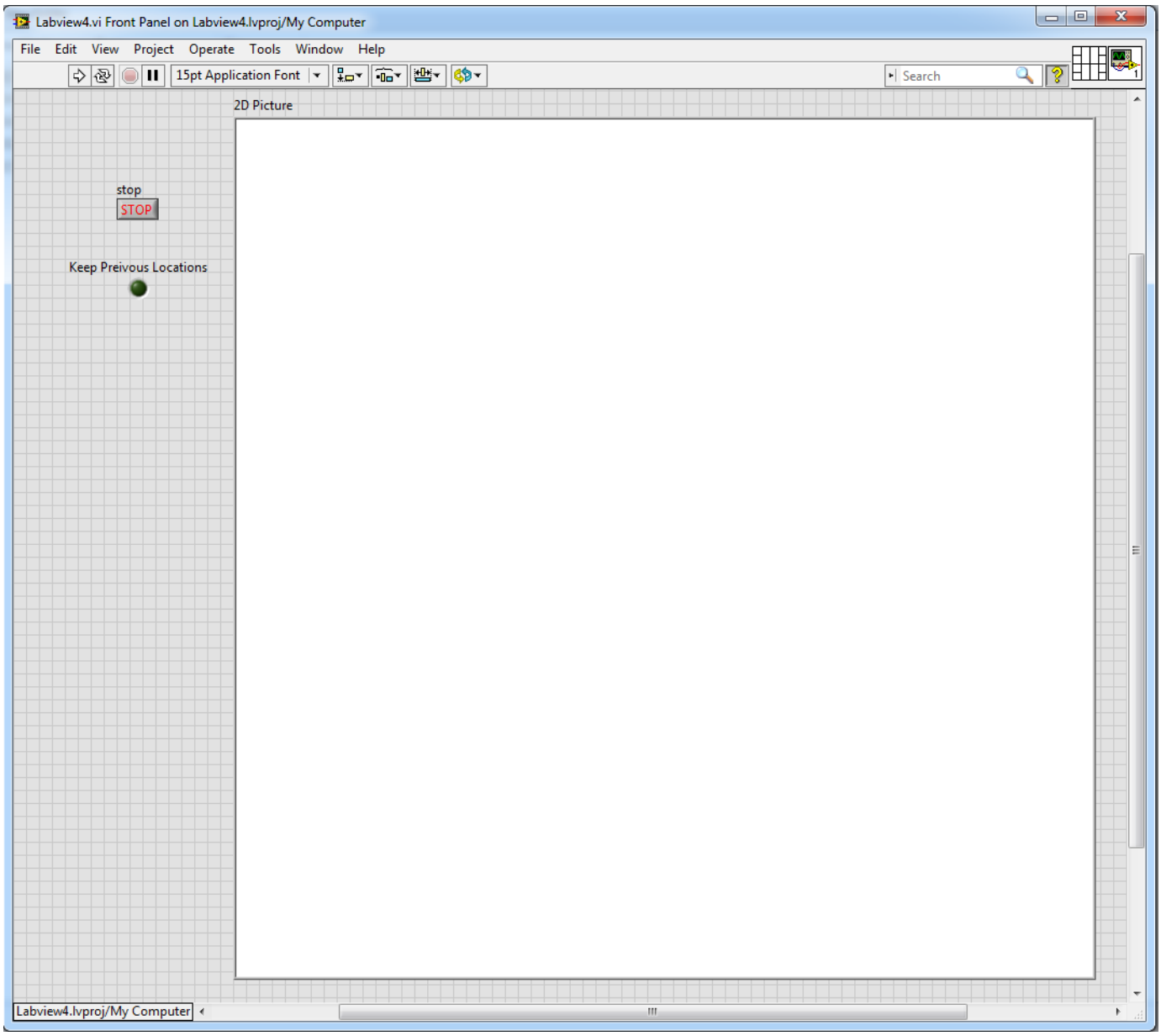
Programming → Graphics & Sound → Picture Functions → Draw Line

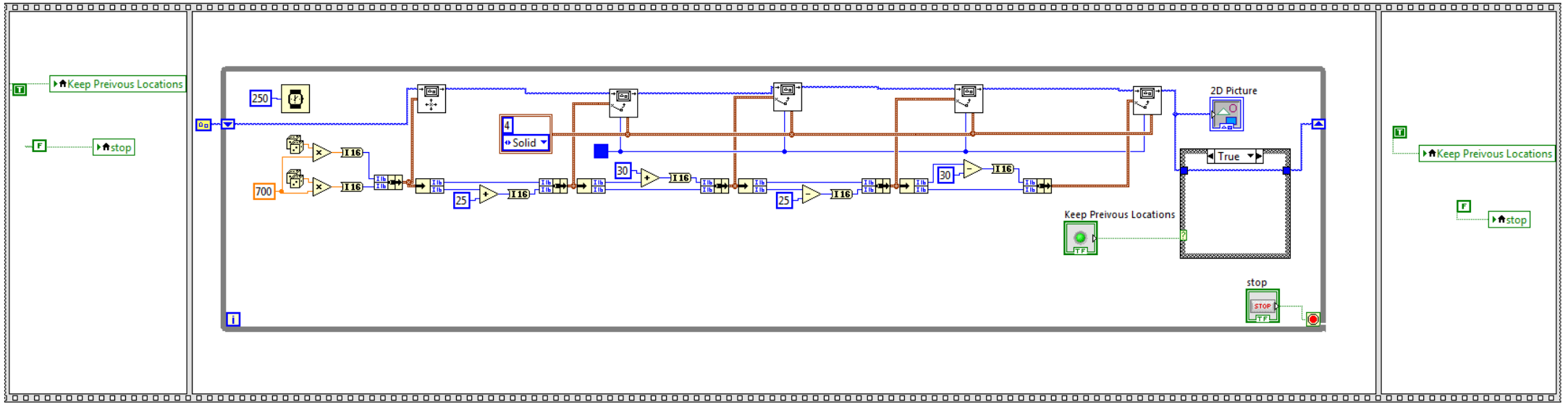
Programming → Graphics & Sound → Picture Functions → Empty Picture

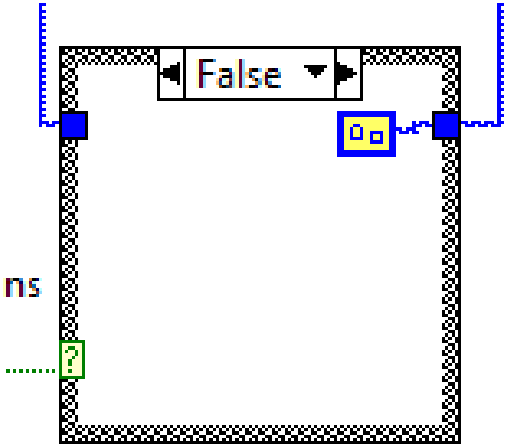
Programming → Numeric → Random Number (0-1)

Programming → Numeric → Conversion → I16

Programming → Cluster, Class, & Variant → Unbundle and Bundle




Keep Preivous Locations
stop
250
700
I16
25
30
4
Solid
2D Picture
True
Keep Preivous Locations
stop
Keep Preivous Locations
stop


False
ns

False Case

Exercise 2

For this exercise, I would like to give you an introduction to communicating to and from LabVIEW using the TCP Ethernet protocol. The goal of the VI is to take two double precision numbers and combine them into a single string. This string is then sent to an Echo Server with the "TCP Write" command. (MATLAB has a nice simple Echo Server that we are going to use.) Then the Echo Server echoes back the same string. We receive this string with the "TCP Read" command and imagine that it is different information then we just sent. Finally, we use the "Scan From String" command to pull the two real numbers out of the string and display them.

Copy this VI and demonstrate it working. *The first time you try to write data across a TCP socket from an application the Windows Firewall will pop up with dialogs asking for permissions. Make sure to check all boxes in those pop-up windows allowing public and private networks.* Because we are using this echo server, you may find it easier to run this LabVIEW application on the lab PCs instead of your laptop even if you have MATLAB installed on your laptop. If things to do work on your laptop, give the lab PCs a try. You will need to find the IP of your computer. Easiest way to do so is to run `ipconfig` from a command prompt. Make sure to find the IP of the main Ethernet connection. On the lab PCs it is usually the first one listed. This is the IPv4 address you will type into the text box entering the "TCP Open Connection" block.

Then before running the LabVIEW application, open MATLAB and type the command `echotcpip("on", 12321)`. 12321 is the port the echo server will be listening to. *Later if you want to turn off the echo server type the command `echotcpip("off")` to turn off the server*. Once the echo server is running in MATLAB , you can run your LabVIEW application. Type in some real numbers into the two Send Values and then click "SEND". Play with different numbers and see that you are getting back the same numbers. When you are done, you can click the "STOP" button.

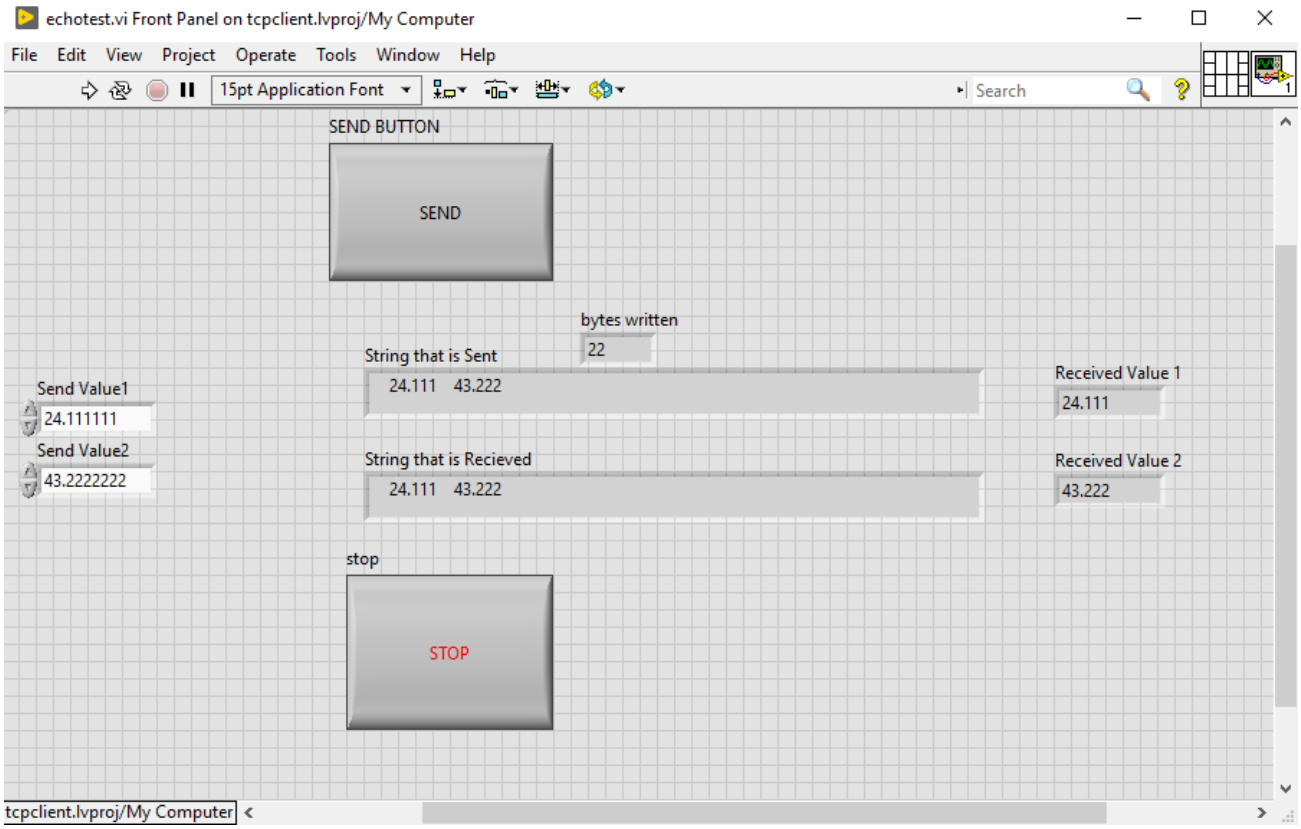
Location of blocks you may have not used for the Block Diagram view:

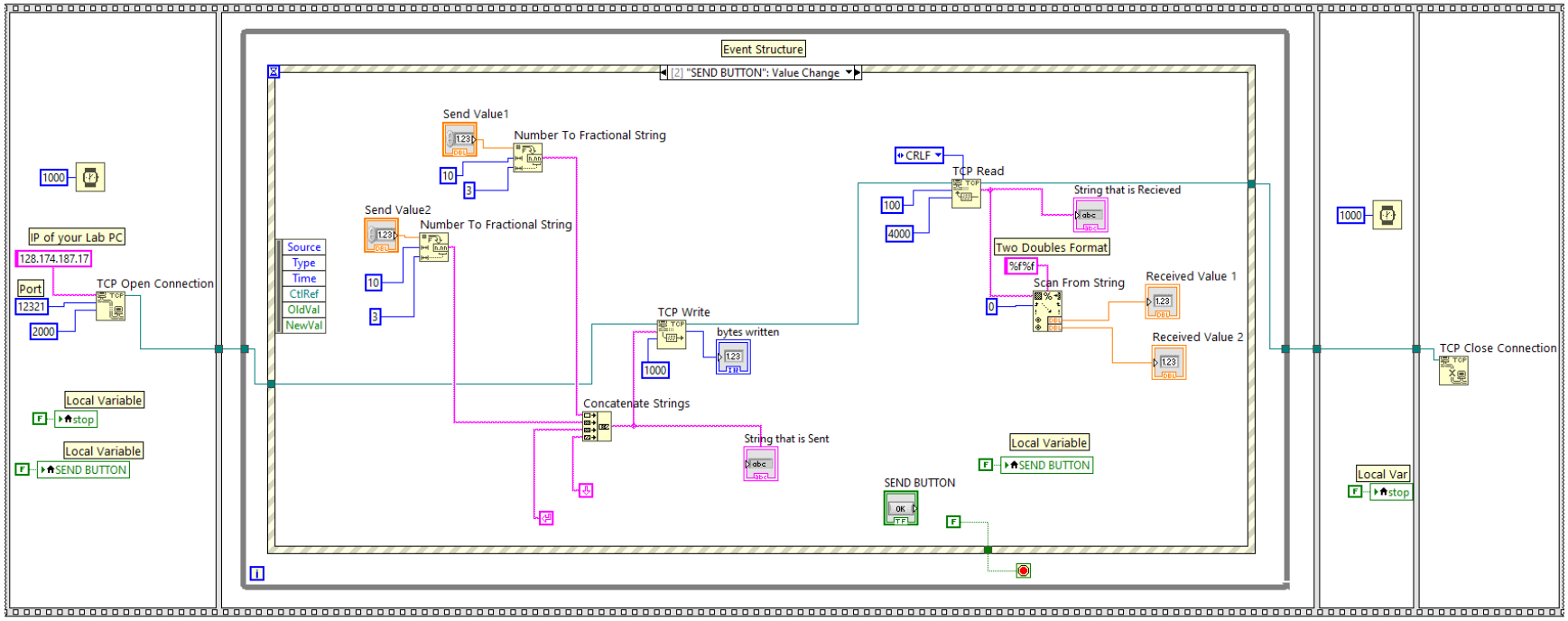
String → Carriage Return Constant and String → Line Feed Constant and String → Scan From String

String → Number/String Conversion → Number To Fractional String

Structures → Local Variable

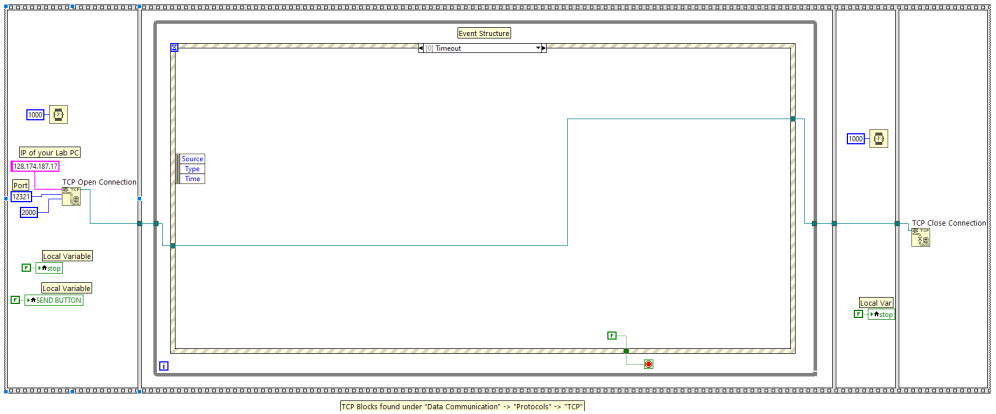
Data Communication → Protocols → TCP → "TCP Open Connection" and "TCP Read" and "TCP Write" and "TCP Close Connection"


echotest.vi Front Panel on tcpclient.lvproj/My Computer
File Edit View Project Operate Tools Window Help
15pt Application Font
Search
SEND BUTTON
SEND
bytes written
22
String that is Sent
24.111 43.222
Send Value1
24.111111
Send Value2
43.2222222
String that is Recieved
24.111 43.222
Received Value 1
24.111
Received Value 2
43.222
stop
STOP
tcpclient.lvproj/My Computer



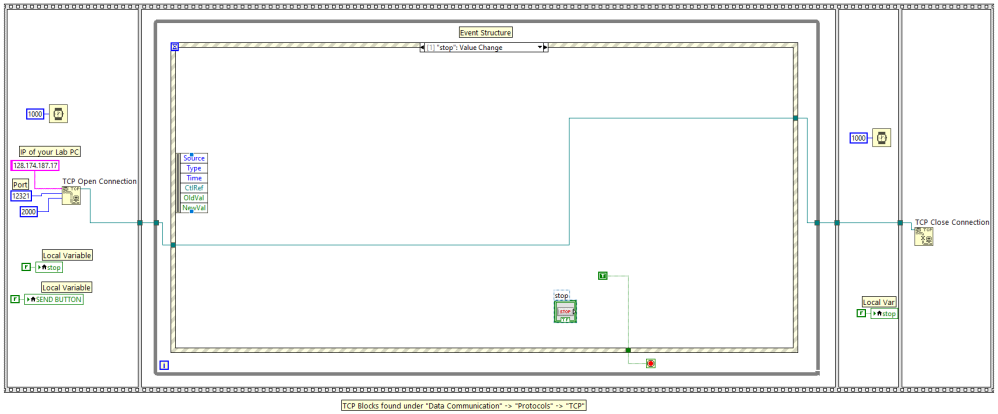
TCP Blocks found under "Data Communication" -> "Protocols" -> "TCP"

Showing the Timeout Event



TCP Blocks found under "Data Communication" -> "Protocols" -> "TCP"

Stop: Value Change Event



TCP Blocks found under "Data Communication" -> "Protocols" -> "TCP"

ME 461 LabVIEW #3

Due October 31st 2:00 PM

This task will be performed on a lab bench robot. Remember to power your robot with the programmable power supply with 12V, 3A.

Most of the work for this final task is to create the LabVIEW program pictured below. This LabVIEW program is an extension to the programs you created in LabVIEW assignment #2. If you have not completed that assignment, I recommend you finish that assignment first.

In LabVIEW assignment #2, you were shown how to draw lines/squares in a Picture Box. In the second exercise you were shown how to communicate using the TCPIP protocol over Ethernet. The TCPIP communication talked to an Echo Server that simply sent back whatever text was sent to it.

For this task you are going to take the experience you gained from LabVIEW assignment #2 and create (copy pictures below) a LabVIEW program that receives eight floating point numbers from your robot's red (F28379D) board at the periodic rate of every 250ms. These eight numbers are displayed in text boxes in your LabVIEW application. In addition, the first two numbers sent are assumed to be the X and Y coordinates of your robot car. The below given C code varies these X and Y points as if the robot was traveling in a circle. In Lab 6 and Lab 7, you will use this program to display in the LabVIEW Picture Box the actual X and Y location of your robot.

In order to have the robot send TCPIP Ethernet data wirelessly to LabVIEW, there is an addition microcontroller on the robot called the ESP32 that has Wi-Fi built in. The TMS320F28379D processor does not have Wi-Fi capability so the ESP32 needs to be added for this communication. You will see in the steps below that you will need to "telnet" into the ESP32 processor which is running an operating system called Nuttx. Easiest way to explain Nuttx is that it is a miniature Linux that can run on slower and small memory capacity microcontrollers. Once you telnet into Nuttx, you will run a C program that I wrote called "tcpLVCOM". This program waits for data either coming wirelessly from LabVIEW or coming serially through a UART serial port connected to the F28379D's UARTD (SCID). Summarizing the communication:

- If you click "Send" in your LabVIEW program, it sends eight floating point numbers over Ethernet/Wi-Fi to the tcpLVCOM application running in Nuttx. When tcpLVCOM receives the eight floats, it then sends the numbers to the red board over UART.
- Also, the given test code below sends eight floating point numbers through UARTD to the tcpLVCOM application every 250ms. When tcpLVCOM receives this new data over UART, it sends this data to LabVIEW over the same TCPIP/Wi-Fi port. Then these eight numbers are displayed in text boxes and the

first two numbers are also used to change the X and Y location of the robot that is drawn as a square in the LabVIEW picture box.

Perform the following steps:

1. Using LabVIEW exercise #2 as a guide, copy the LabVIEW program given in the below two pictures. Notes for completing this exercise are as follows:

    a. Your robot's IP number is written on your robot car. The numbers written on our 12 robots range from 51 to 62. This number is the last number in your robot's IP 192.168.1.<yournumber>. If your robot has 55 written on it, your IP is 192.168.1.55. Also note that this new program has two while loops. One while loop is for sending eight floats to your robot and the other while loop is for receiving eight floats from your robot.

    b. The Send code / Event structure from LabVIEW exercise #2 is a good starting point for the Send of this exercise reminding you about the Timeout Event and Stop Event. In this case however, the Send event structure only sends data to the robot. The receiving of data happens in the second while loop. Eight floats, converted to text, are sent and I add these \FD and \FF characters as start and stop characters. Make sure to right click on these pink string constants and select "\ Display Code." Also make sure there is only one "\" back slash in the string constant. This tells LabVIEW to send the hexadecimal number 0xFD and 0xFF instead of the ASCII characters 'F' and 'D'.

    c. Make sure the Mechanical Action of both the Send and Stop buttons is set to "Switched When Pressed" by right clicking on the buttons.

    d. After creating the "Size of Rectangle" control, set it to 20 and then right click on it and select "Data Operations->Make Current Value Default." This will reload 20 in this variable the next time you open this program.

    e. To test your LabVIEW program you need to finish the remaining steps to setup both the red board and the ESP32.

2. In CCS create a new "LABstarter" project and rename it. Much of the code for communicating to and from LabVIEW, Nuttx, and the red board is given to you already in LABstarter. *If you are interested, the bulk of the communication code for the red board is in the UARTD receive interrupt function "RXDINT_recv_ready" in F28379dSerial.c*. The remaining code you need is given here:

    a. Cut and paste the following global variables towards the top of your project's C file. For now you can use these variables below. But in the future you may want to give the eight LV variables more descriptive names.

```
float printLV1 = 0;
```

```
float printLV2 = 0;
float printLV3 = 0;
float printLV4 = 0;
float printLV5 = 0;
float printLV6 = 0;
float printLV7 = 0;
float printLV8 = 0;
extern uint16_t NewLVData;
extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
extern LVSendFloats_t DataToLabView;
extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
extern uint16_t newLinuxCommands;
extern float LinuxCommands[CMDNUM_FROM_FLOATS];
```

b. Then setup Timer 0's interrupt function to be call every 1 ms.

c. Copy the following code into Timer 0's interrupt function before the line of code that increments "numTimer0calls." Look as this code. "NewLVData" is a flag variable that is set for you when new data has been received from LabVIEW/Nuttx. When "NewLVData" is equal to one, copy the eight received floats into global variables you can use. Then also every 250 ms., this code sends eight floats up to Nuttx/LabVIEW. Look at the eight things we are sending. I use sin() and cos() and numTimer0calls to simulate X, Y coordinates. Then, so that we can see that the six other variables are being sent and received, I assigned them different fractions of "numTimer0calls." In the future you can change this code to upload the robot's actual X, Y coordinates and other data you want to send to LabVIEW.

```
if (NewLVData == 1) {
        NewLVData = 0;
        printLV1 = fromLVvalues[0];
        printLV2 = fromLVvalues[1];
        printLV3 = fromLVvalues[2];
        printLV4 = fromLVvalues[3];
        printLV5 = fromLVvalues[4];
        printLV6 = fromLVvalues[5];
        printLV7 = fromLVvalues[6];
        printLV8 = fromLVvalues[7];
}

if((numTimer0calls%250) == 0) {
        DataToLabView.floatData[0] = 3.0*sin(2*PI*.05*numTimer0calls*0.001);
        DataToLabView.floatData[1] = 3.0*cos(2*PI*.05*numTimer0calls*0.001);
        DataToLabView.floatData[2] = (float)numTimer0calls*.001;
        DataToLabView.floatData[3] = 2.0*((float)numTimer0calls)*.001;
        DataToLabView.floatData[4] = 3.0*((float)numTimer0calls)*.001;
        DataToLabView.floatData[5] = (float)numTimer0calls;
        DataToLabView.floatData[6] = (float)numTimer0calls*4.0;
        DataToLabView.floatData[7] = (float)numTimer0calls*5.0;
        LVsenddata[0] = '*'; // header for LVdata
        LVsenddata[1] = '$';
        for (int i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
                if (i%2==0) {
                        LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
                } else {
                        LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
                }
        }
serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
}
```

d. In main()'s while(1) loop change your serial_printf statement so that it prints all eight values sent from LabVIEW.

```
serial_printf(&SerialA,"%.3f,%.3f,%.3f,%.3f,%.3f,%.3f,%.3f,%.3f\r\n",
printLV1,printLV2,printLV3,printLV4,printLV5,printLV6,printLV7,printLV8);
```

e. Build and download this code to your robot. Power the robot with 12V, 3A from the programmable power supply in lab. In CCS run your red board's code and open a Tera Term session to see what is printing. You should see the eight variable values sent from LabVIEW printed to Tera Term. They should be all zero as we have not setup the communication yet.

f. When the robot is powered on, the ESP32 board is also powered. It takes about 10 to 15 seconds for Nuttx to boot on the ESP32 board. After waiting 15 seconds, open a CMD prompt in Windows and test that Nuttx is ready by pinging your robot's IP:

`ping 192.168.1.<your robot's IP number>` (i.e., `ping 192.168.1.55`)

You should receive a response. Press "<CTRL> + C" to end the ping. If you do not receive a response with ping, check that you typed the correct IP. Also, you can try pressing the small "EN" button on the robot's ESP32 board, waiting 15 seconds, and trying again.

g. Then from the Windows CMD prompt connect to Nuttx over telnet by typing:

`putty -telnet 192.168.1.<yourRobotsIPnumber>`

This should give you a Nuttx Shell, nsh, prompt. At this prompt type:

`tcpLVCOM`

The tcpLVCOM application is now waiting for your LabVIEW program to connect.

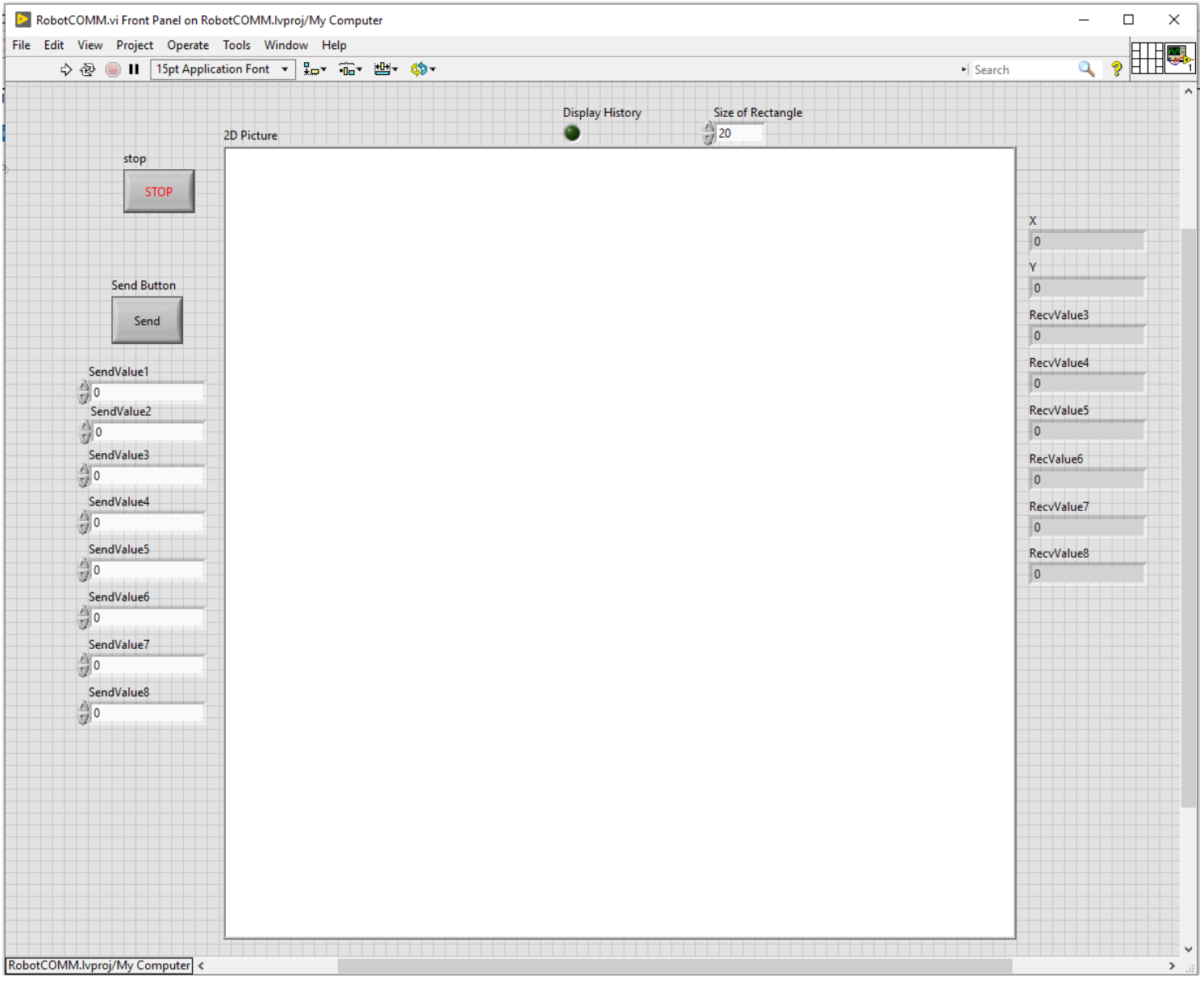
h. Make sure your red board's code is running in CCS and printing to Tera Term.

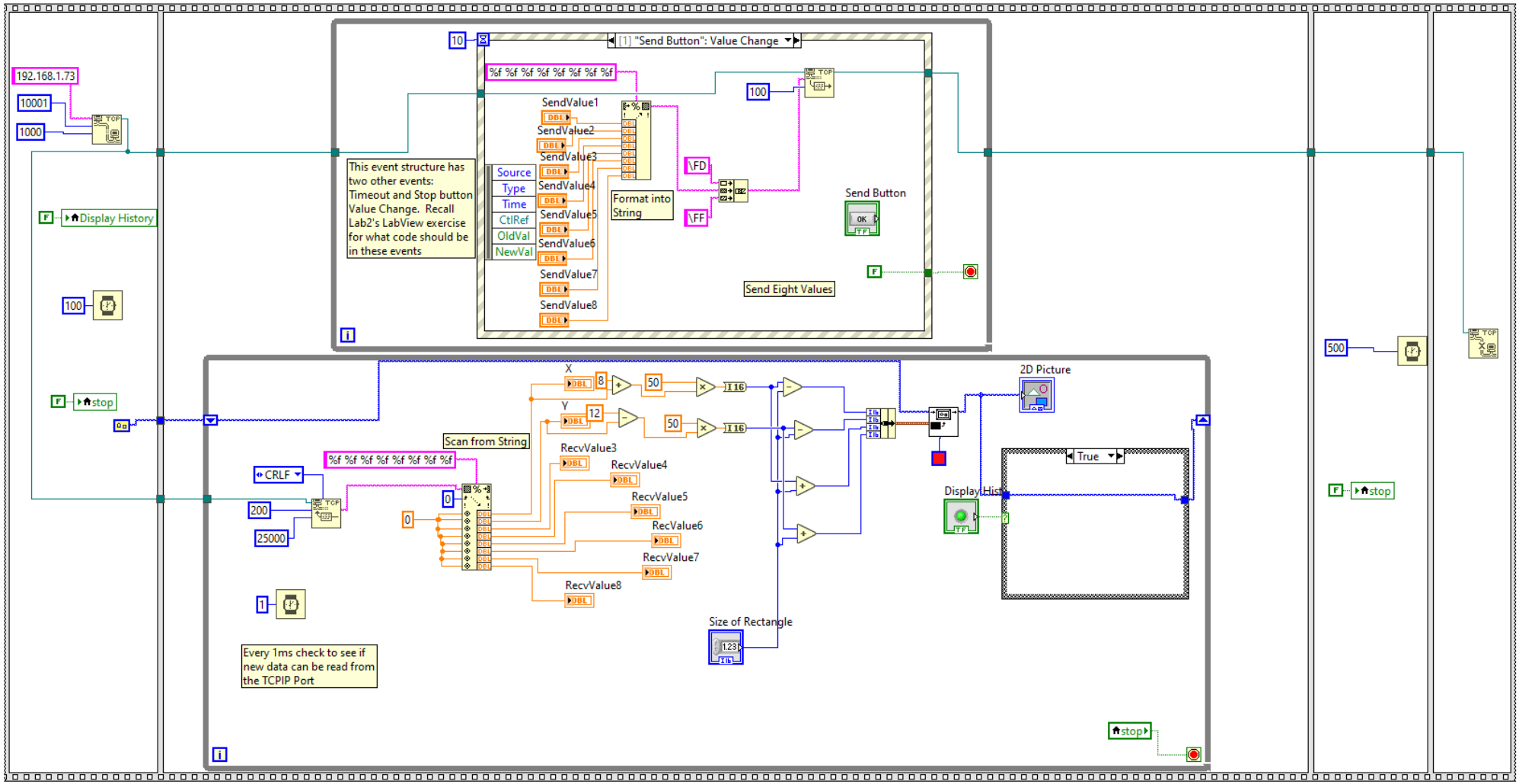
i. Run your LabVIEW program.

j. If everything is working, you should see your robot square moving in the Picture Box of LabVIEW. In the Nuttx shell, the values being received from the red board should be printing.

k. To test, download from LabVIEW to the robot. Type different floating point values in the eight send variables and press the "Send" button. You should see the printed values in Tera Term change to the values you typed in LabVIEW.

l. **Demo this working to your TA or Instructor** and/or continue to the last steps listed after the two below pictures.


RobotCOMM.vi Front Panel on RobotCOMM.lvproj/My Computer
File Edit View Project Operate Tools Window Help
15pt Application Font
Search
Display History
Size of Rectangle
20
2D Picture
stop
STOP
Send Button
Send
SendValue1
0
SendValue2
0
SendValue3
0
SendValue4
0
SendValue5
0
SendValue6
0
SendValue7
0
SendValue8
0
X
0
Y
0
RecvValue3
0
RecvValue4
0
RecvValue5
0
RecvValue6
0
RecvValue7
0
RecvValue8
0
RobotCOMM.lvproj/My Computer


192.168.1.73
10001
1000
Display History
100
stop
10
[1] "Send Button": Value Change
%f %f %f %f %f %f %f %f
SendValue1
SendValue2
SendValue3
SendValue4
SendValue5
SendValue6
SendValue7
SendValue8
Source
Type
Time
CtlRef
OldVal
NewVal
This event structure has two other events: Timeout and Stop button Value Change. Recall Lab2's LabView exercise for what code should be in these events
Format into String
\FD
\FF
100
Send Button
OK
Send Eight Values
X
Y
8
12
50
50
I16
I16
2D Picture
Scan from String
%f %f %f %f %f %f %f %f
CRLF
200
25000
0
0
RecvValue3
RecvValue4
RecvValue5
RecvValue6
RecvValue7
RecvValue8
1
Every 1ms check to see if new data can be read from the TCPIP Port
Size of Rectangle
True
Display Hist
stop
500
stop

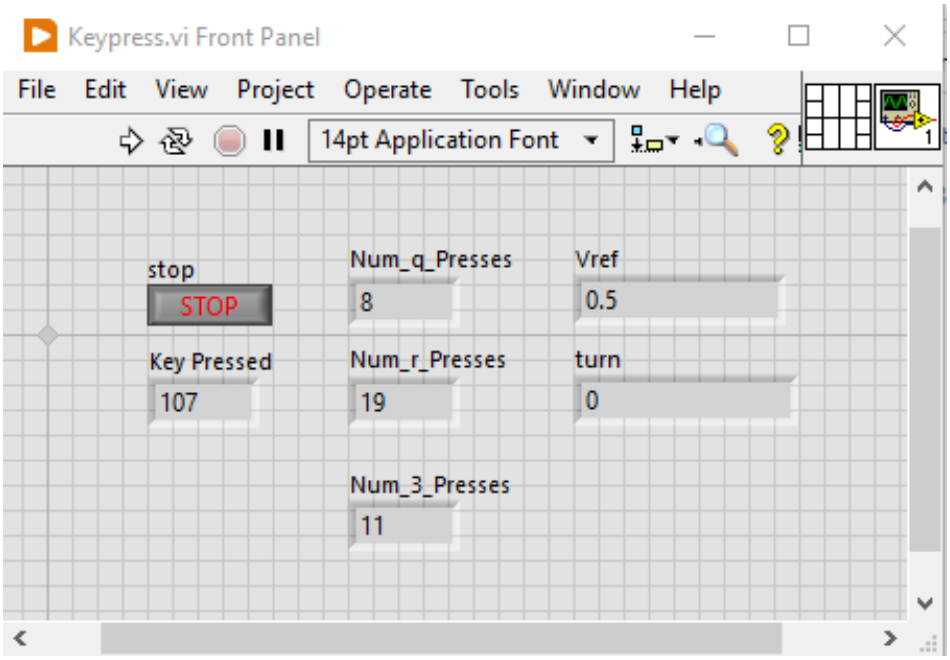
m. Add one more feature to this LabVIEW program that will make steering around your robot wirelessly a bit easier in Lab 6 and Lab 7.

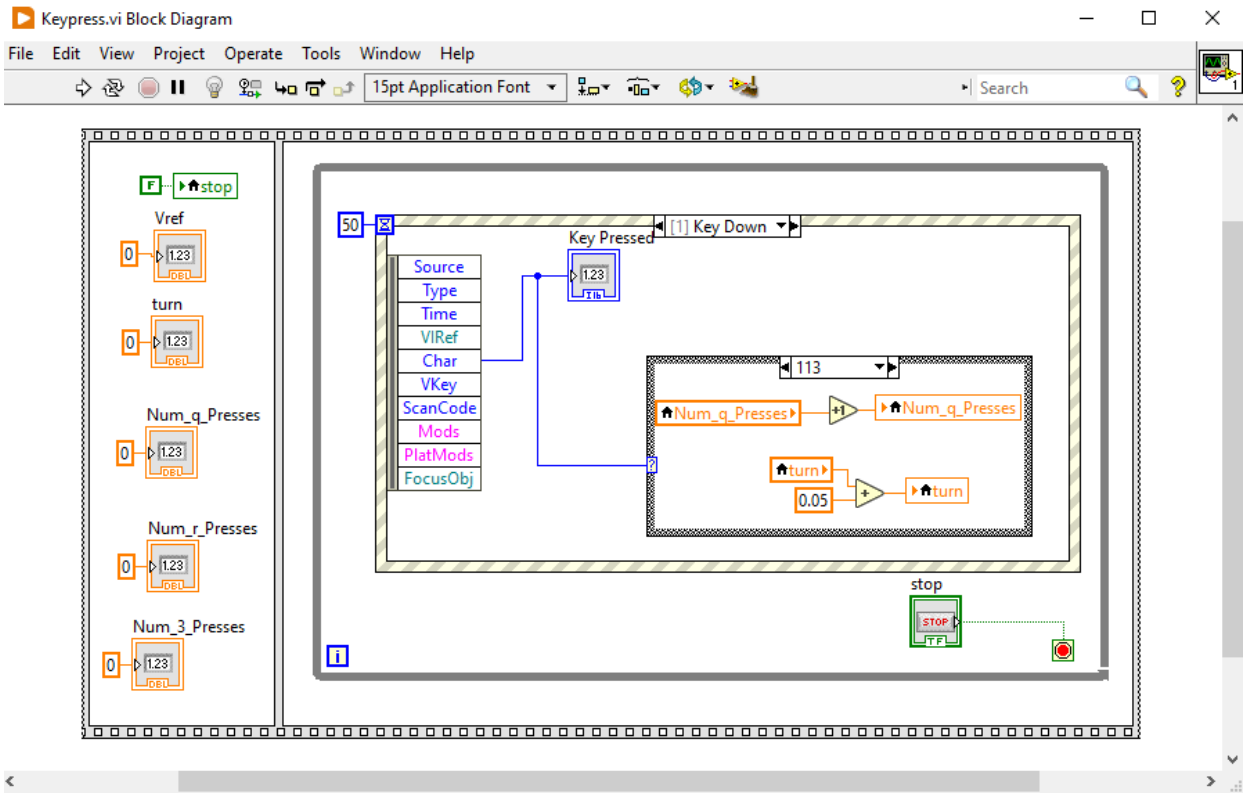
The Event structure has an event <This VI> → Key → Key Down. This event is run when your LabVIEW program has focus (mouse click the Front Panel View) and you press any key. Which key was pressed is given to you by the "Char" variable created by the event structure. So, using the below sample LabVIEW application, add to the above Event structure a Key Down event.

Copy most of what is in the "Send Button": Value Change event into this new Key Down event. BUT NOTE: you will have to use local variables to get the values of SendValue3 through SendValue8. Instead of using local variables for SendValue1 and SendValue2, write the value of "Vref" (Use a Local Variable block) to the first "Format Into String" input and "turn" to the second "Format Into String" input. You will probably have to make your Event structure a bit bigger to fit all the below code into your Key Down event.

Looking at the below example code, notice that when you press "q" (ASCII 113) the value of 0.05 is added to "turn". If "r" (ASCII 114) is pressed 0.05 is subtracted from "turn". If "3" is pressed a value of 0.1 is added to "Vref". If any other key is pressed, "0, Default", "turn" is set to 0 and "Vref" is set to 0.5. Add this last feature to your application and then **demo to your TA or Instructor** that your LabVIEW application receives values from the Red Board (through ESP32) and that both your Send Button event and Key Down event work to send new values to the Red Board.

Example Application to show you how to use the "<This VI> → Key → Key Down" Event. It is up to you if you want to build this small application first before you add the Key Down event to your Red Board interface App above.



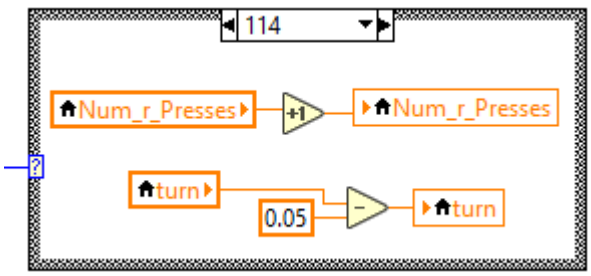


The three variables, "Num_q_Presses", "Num_r_Presses", and "Num_3_Presses" are not needed in your code. These variables are just incremented by one each time the Key Down event is called and that specific key was pressed.

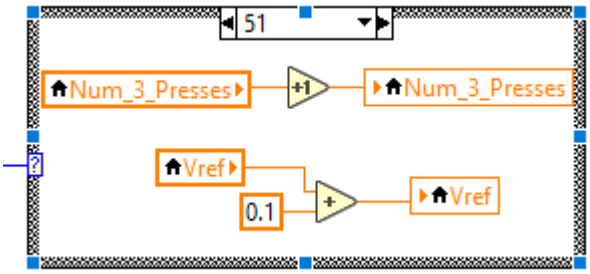
Notice how the Event Structure knows which key is pressed by looking at the event variable "Char". "Char" is an I16 variable, so it is the ASCII value for the key pressed: "q" = 113, "r" = 114, "3" = 51. Using a Case Structure with four items, the below code changes "Vref" and "turn". If "q" is pressed turn = turn + 0.05. If "r" is pressed turn = turn – 0.05. If "3" is pressed Vref = Vref + 0.1. If any other key is pressed, turn = 0, Vref = 0.5.

Key press case examples:

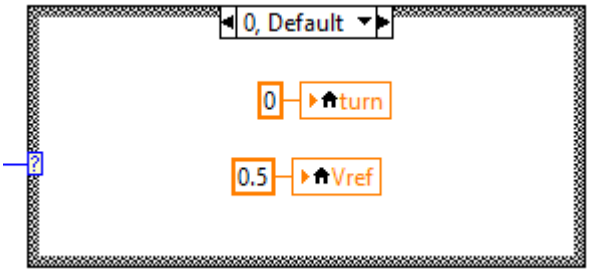
- Case for "r" being pressed:



- Case for "3" being pressed:



- Case for Any Other Key being pressed:



ME 461 LabVIEW #4

These are optional Exercises for LabVIEW if You are Interested in Learning More LABVIEW Programming.

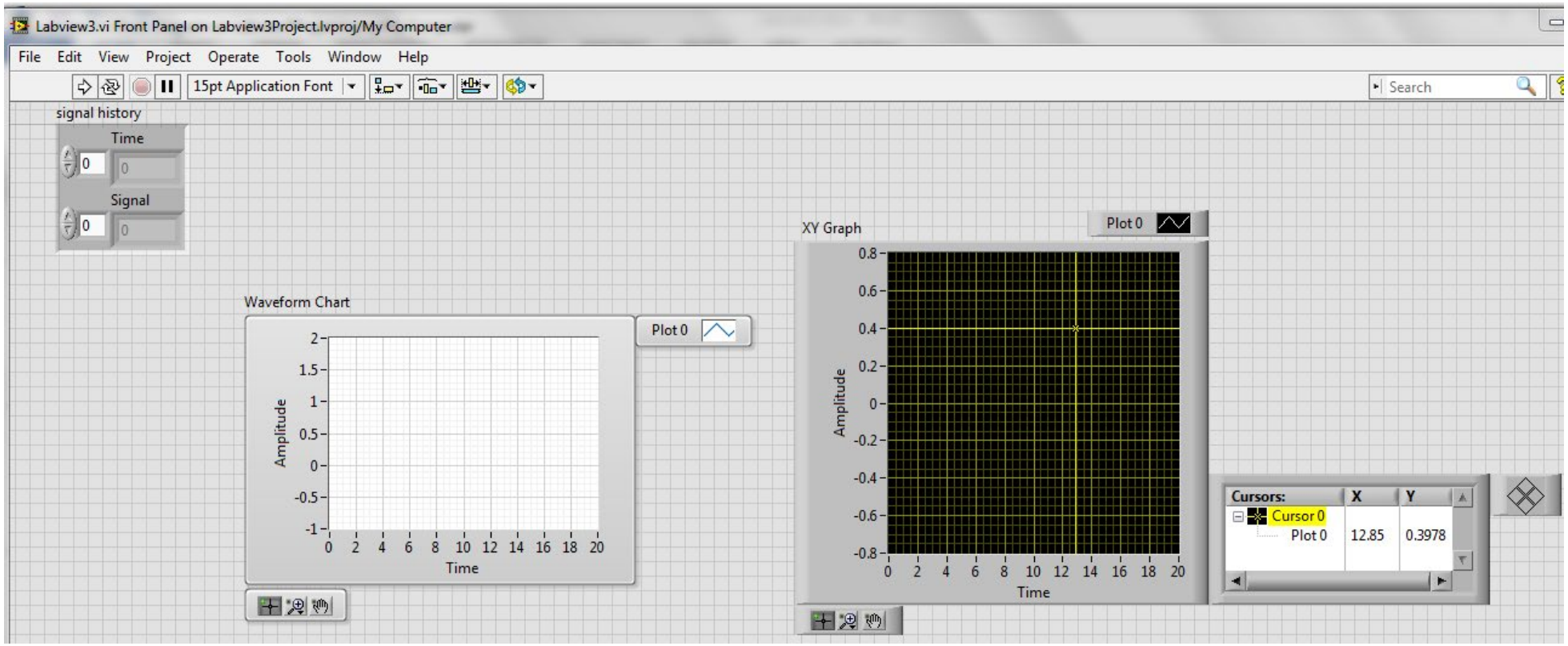
Exercise 1

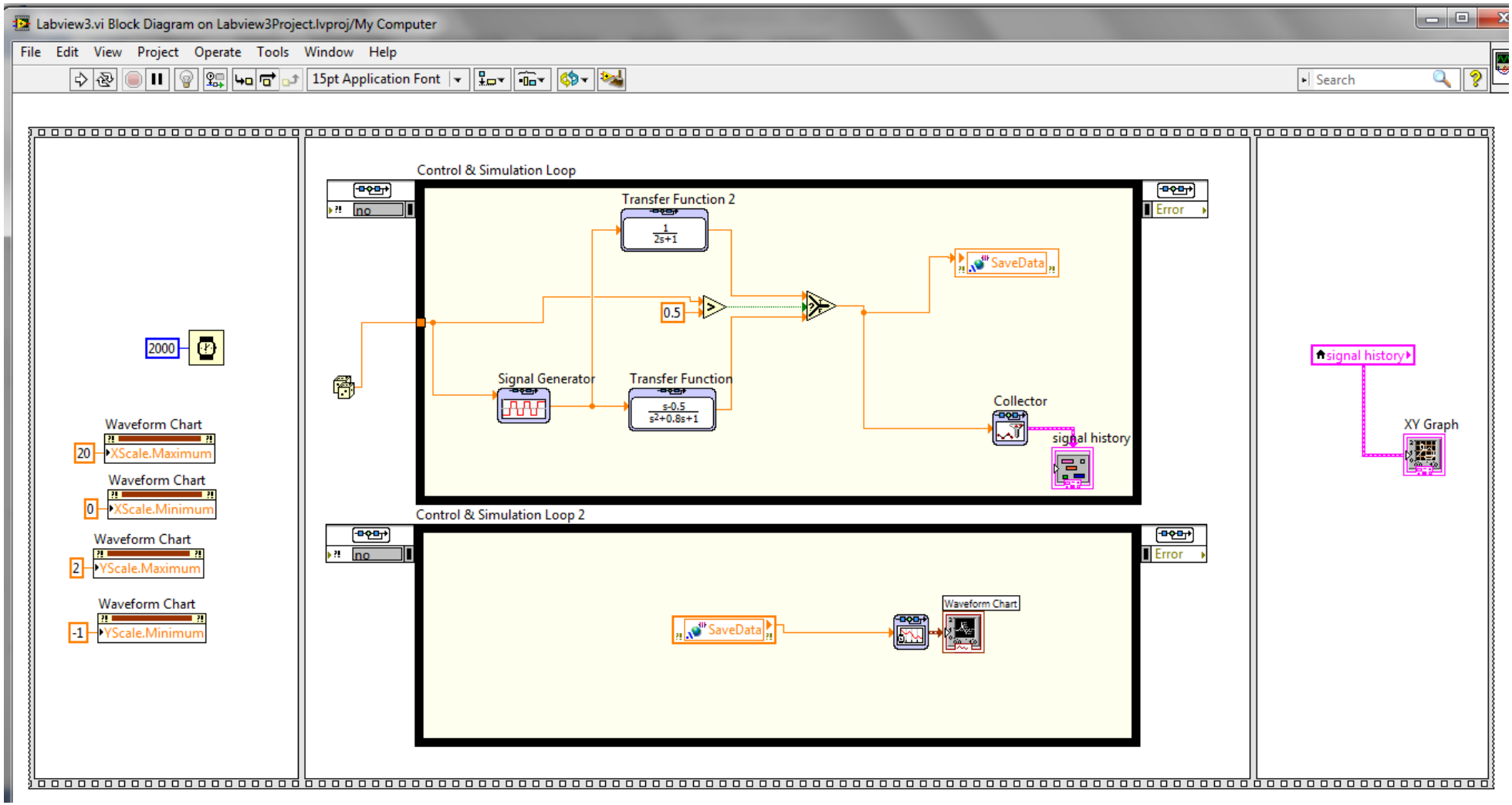
I would like you to go through the "Basics of Control Design and Simulation" tutorial. It can be found at http://coecsl.ece.illinois.edu/me461/labs/NI-Tutorial-BasicsofCDS.pdf . When you are done with this tutorial you will have saved three files: "My Controls Example VI.vi", "My Controls Example With Math Script VI.vi" and "My Control and Simulation Example VI.vi" Demo all three of these Vis working for the first part of your check off.

The last VI you build, "My Control and Simulation Example VI.vi", uses the Control and Simulation Loop. You should be familiar with MathWorks Simulink from previous classes like ME340 and ME360. You can think of the blocks you place inside the Control and Simulation Loop the same as blocks in a Simulink simulation.

Exercise 2

For this next exercise I would like you to reproduce the VI shown in the pictures below. Start out by creating a LabVIEW Project. Inside this project, add a single VI. The reason I want you to use a project is that I would like you to try out using a "shared variable" and these are created in a separate .lvlib file that will also become part of your project. I will give some pointers and instructions below, but they will not be necessarily in the exact order you need to perform the steps or completed steps. In other words, you may need to do a bit of your own hunting to find the necessary blocks and other parts of the program and use LabVIEW help to figure out how to use/wire certain blocks. Many of the blocks I used below come from the "Control Design & Simulation → Simulation" section of LabVIEW

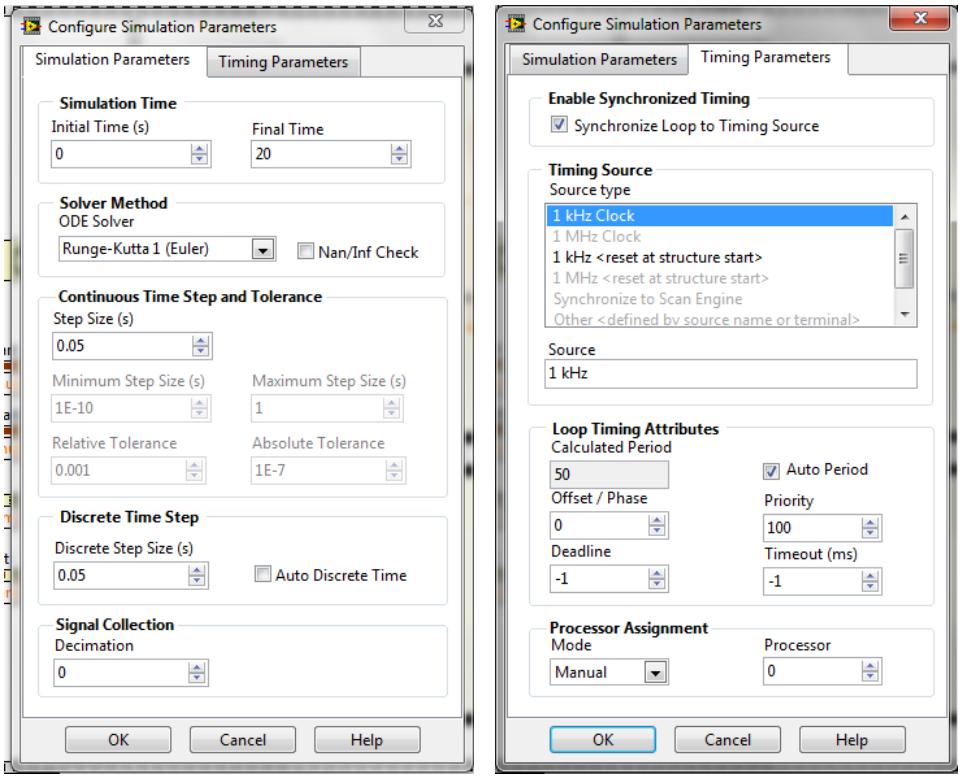

Labview3.vi Front Panel on Labview3Project.lvproj/My Computer
File Edit View Project Operate Tools Window Help
15pt Application Font
Search
signal history
Time
0
0
Signal
0
0
Waveform Chart
Plot 0
Amplitude
Time
XY Graph
Plot 0
Amplitude
Time
Cursors:
X
Y
Cursor 0
Plot 0
12.85
0.3978


Labview3.vi Block Diagram on Labview3Project.lvproj/My Computer
File Edit View Project Operate Tools Window Help
15pt Application Font
Search
Control & Simulation Loop
no
Error
Transfer Function 2
1
2s+1
SaveData
0.5
Signal Generator
Transfer Function
s-0.5
s²+0.8s+1
Collector
signal history
2000
Waveform Chart
20
XScale.Maximum
Waveform Chart
0
XScale.Minimum
Waveform Chart
2
YScale.Maximum
Waveform Chart
-1
YScale.Minimum
Control & Simulation Loop 2
no
Error
SaveData
Waveform Chart
signal history
XY Graph

Pointers/Instructions

1. Why am I using two Control and Simulation Loops? I am splitting the simulation into two parts. One to run the simulation and the second to plot the data. When you start using the simulation loop on the "MyRIO" development board you will find that to achieve consistent sample periods you will need to divide processing between the two processor cores of the MyRIO. So, to prepare you for this I'm showing you here how we are going to plot data. The simulation's "y" output is stored into a "Shared Variable" that has a Real-Time FIFO enabled. In the block diagram view below it is the orange "SaveData" blocks. We will set up one of the simulation loops to run on one processor core and the other simulation loop to run on another processor core.

2. After you add the two simulation loops, double click on their parameters tab in their top left corner which will bring up the Configure Simulation Parameters window. Set it up as in the pictures below. For the second Simulation Loop, set it up as the same except for the Processor Assignment. Set the processor as "1".



3. In your project window, right click on the "My Computer" under your project name and select New → Variable. Give the variable a name. Make the variable type "Single Process" and data type "Double". Click on the RT FIFO item and check the item "Enable RT FIFO". The shared variable is stored in a library .lvlib file. Save this library file with a name of your choosing.

4. To add the shared variables to your application, simply drag the shared variable from the project window into your block diagram window. Then in your Block Diagram, right click on the shared variable and select the appropriate "Access Mode."

5. Block Locations:

    a. Use the Flat Sequence to order when items of your program run. Programming → Structures.

    b. Transfer Function: Control Design & Simulation → Simulation → Continuous The transfer function inside the block can be whatever you choose.

    c. Signal Generator: Control & Simulation → Simulation → Signal Generation

    d. Notice that I set up the Signal Generator to have a terminal input. I want the Amplitude of the Signal Generator to change each run by using the random number generator. If you double click on the signal generator and select the amplitude parameter you can change its parameter source to "terminal."

    e. Collector: Control & Simulation → Simulation → Utilities. The collector is optimized for the simulation loop and is used to collect data for plotting later or even offline. To create the Collector's output variable that I called "signal history", hover your mouse just to the right of the Collector and right click. Select Create →Indicator.

    f. The Waveform Chart and its accompanying block to its left is called a "SimTime Waveform". Find it under Control & Simulation → Simulation → Graph Utilities.

    g. Select and Greater: Programming → Comparison. I am showing you the Select block so you can see how to choose between two signals and only route one of them to a desired block.

    h. Dice (Random Number): Programming → Numeric.

    i. Local Variable: Programming → Structures.

    j. To add the XY Graph to your program you must switch to the Front Panel window. From there right click and find the XY Graph in Modern → Graph.

    k. Property Nodes XScale.Minimum, XScale.Maximum, YScale.Minimum, YScale.Maximum. Property Nodes allow you to change properties of a front panel object (in this case a graph) during your program run. Here I am just setting the range of the X and Y scales. You will see there are a huge number of properties you can change on the fly if you wish. Since there are so many it is hard sometimes to find the correct property to change the item you are interested in. Using Help and trial and error

is the way to finally figure out which property to change. Create these property nodes by right clicking on the Waveform Chart block and selecting Create → Property Node → XScale → Range → Minimum. Do the same for X Max, Y Min, and Y Max. If you right click on the Property Node the top option allows you to change the property to a "Write". (Or "Read" if you need to switch back.)

6. In the Front Panel window, right click on both of the graph objectives and turn on their different "Visible Items". The "Graph Palette" allows you to zoom in and out the graph's data. The "Cursors Legend" is only available in the XY Graph and it creates a cross hair cursor. After you run your program, play around with both the zooming capability and the cursors.

To save your collected data to a file, to load into Excel or MATLAB, you simply right click on the plot and select Export. This allows you to export directly to an Excel file or you can have it copy the data to the clipboard and then you can pull up your favorite editor (I like Notepad++) and paste all the data into that file. Play around with this export feature and make sure you can pull data into Excel or your favorite editor.

Introduction for Ex 3,4,5

For this exercise, I would like you to first reproduce the VI shown in exercise 1. Then, in exercise 2 and 3, create a similar VI that performs the same function as the first using different methods. Exercise 1 utilizes an "if-else" condition, exercise 2 uses a SubVI, and exercise 3 implements the same function in the form of a Formula Node.

Again, for this this LabVIEW start out by creating a LabVIEW Project. Then inside this project, add a single VI. We are using a project this time so that the SubVI you will be creating will be saved in your project. I will give some pointers and instructions below, but you will need to do a bit of your own hunting to find the necessary blocks and other parts of the program and use LabVIEW help to figure out how to use/wire certain blocks. Many of the blocks I used below come from the "Control Design & Simulation → Simulation" section of LabVIEW.

Note: There are a lot of small blocks in the LabVIEW code I give you below for Exercise 1 and 2. I have found if you open the Word document, you can zoom into the code pictures a bit better than the PDF file.

Exercise 3

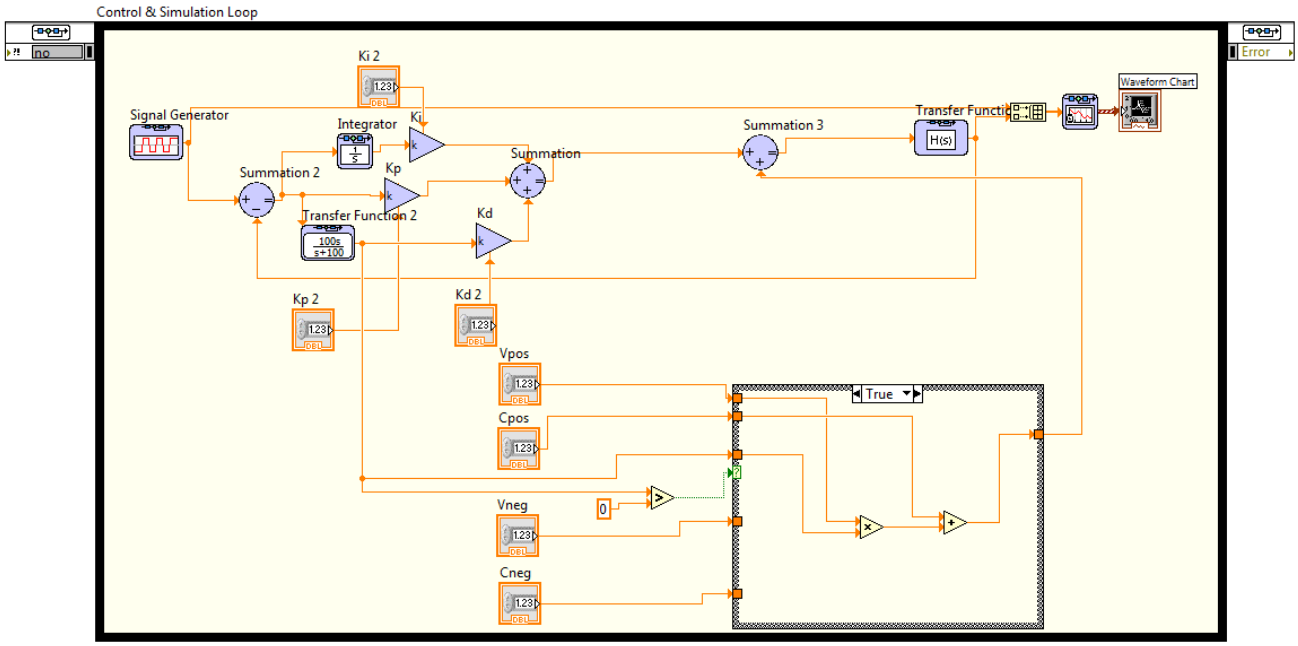
Reproduce the below VI.

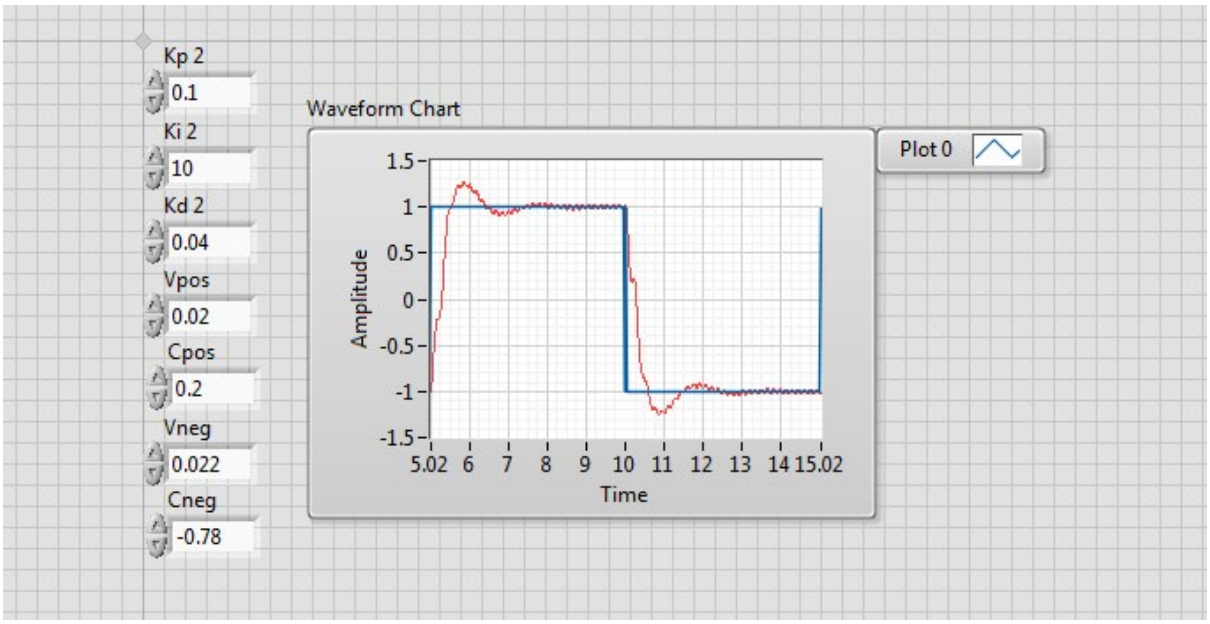
Instructions & tips:

1. The entire program is inside a Simulation Loop. The default simulation settings should work fine for this exercise.
2. You will notice that I am using a Case Structure in this VI. The case structure is implementing the following if statement without explicit code:

```
if (velocity > 0) {
        FricComp = Vpos*velocity + Cpos; // viscous friction*velocity + static friction
} else {
        FricComp = Vneg*velocity + Cneg;
}
```

3. The signal generator is set to square wave with an amplitude of 1 and frequency of 0.1 Hertz
4. Set gain blocks "terminal" mode so that the gain can be changed with a control.
5. We are implementing a PID controller. Gain values are given in the Control Items in the Front Panel window.
6. The plant transfer function is $\frac{1600}{s^3+14s^2+448s+3200}$.
7. $\frac{100s}{s+100}$ is an approximation of a derivative.
8. The graph is a SimTime Waveform block.
9. I am using a Build Array block in front of the SimTime Waveform to allow for plotting two signals on one graph.
10. Run the program with the given constant values below. You should see a similar output to the Waveform Chart shown.



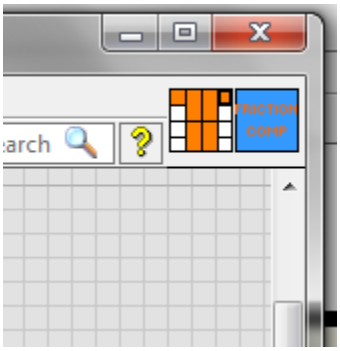


Exercise 4

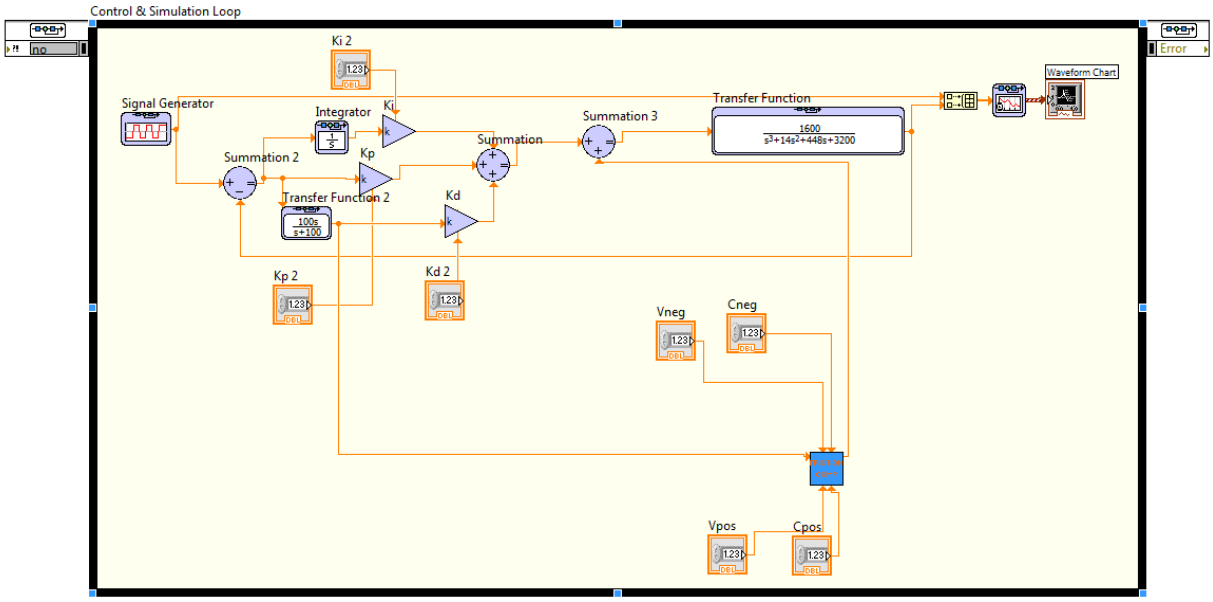
Now, change the Friction Compensation part of the VI into a SubVI. See the following tutorial to help you create this SubVI from the blocks already in your simulation:

https://www.youtube.com/watch?v=zr2qqv1F4kk

Make a simple icon for your VI and connect the inputs and outputs to the pattern shown in the picture below. There is also another picture below of the simulation with the SubVI.



Picture of Icon and Pattern to use



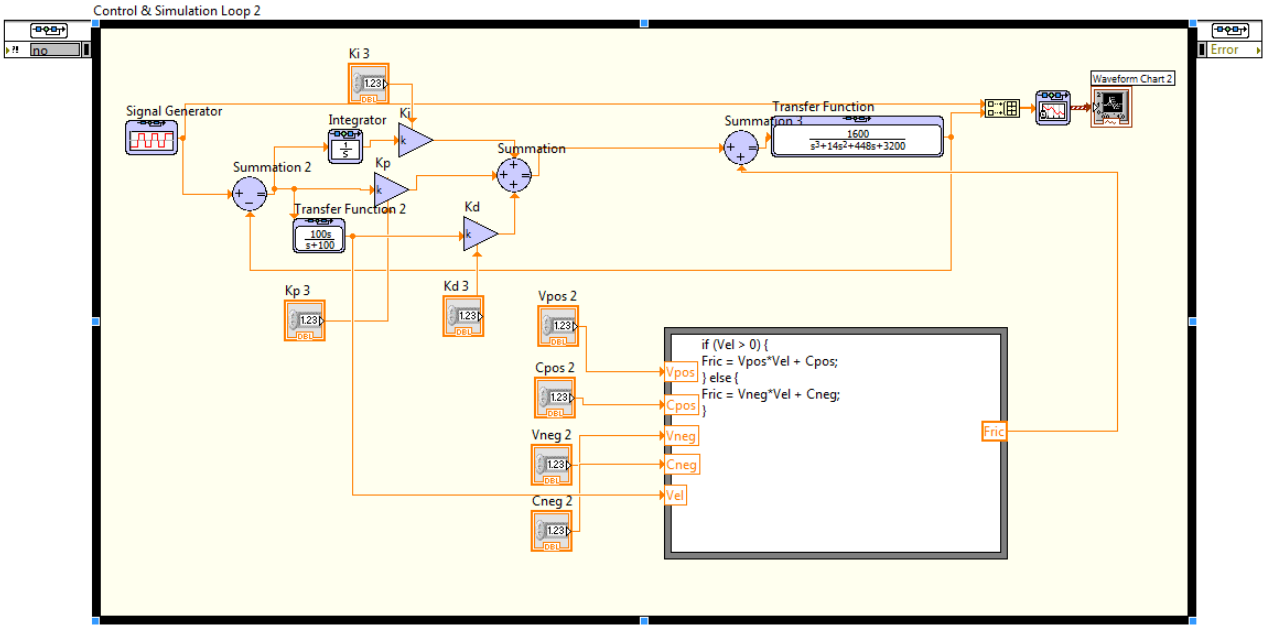
Picture with SubVI

Exercise 5

Now either create a new VI or copy the Simulation Loop and all its contents and run two simulation loops in one VI.

Here in this new VI or new simulation loop, everything is the same except I would like you to replace the SubVI with a Formula Node. A Formula Node allows you to add small amounts of C-code to your LabVIEW program.

To add a variable to a formula node right click on an edge of the formula node and select “Add Input.” To add an output for the formula, right click on an edge and select “Add Output.”


Control & Simulation Loop 2
no
Error
Ki 3
Signal Generator
Integrator
Ki
Summation
Summation 2
Kp
Transfer Function 2
Kd
Transfer Function
Summation 3
1600
s³+14s²+448s+3200
Waveform Chart 2
Kp 3
Kd 3
Vpos 2
Cpos 2
Vneg 2
Cneg 2
if (Vel > 0) {
Fric = Vpos*Vel + Cpos;
} else {
Fric = Vneg*Vel + Cneg;
}
Vpos
Cpos
Vneg
Cneg
Vel
Fric